

Unit-III: Autoencoders in Machine Learning

Autoencoders are a special type of neural networks that learn to compress data into a compact form and then reconstruct it to closely match the original input. They consist of an:

- Encoder that captures important features by reducing dimensionality.
- Decoder that rebuilds the data from this compressed representation.

The model trains by minimizing reconstruction error using loss functions like Mean Squared Error or Binary Cross-Entropy. These are applied in tasks such as noise removal, error detection and feature extraction where capturing efficient data representations is important.

Architecture of Autoencoder

An autoencoder's architecture consists of three main components that work together to compress and then reconstruct data which are as follows:

Autoencoder

1. Encoder

It compress the input data into a smaller, more manageable form by reducing its dimensionality while preserving important information. It has three layers which are:

- **Input Layer:** This is where the original data enters the network. It can be images, text features or any other structured data.
- **Hidden Layers:** These layers perform a series of transformations on the input data. Each hidden layer applies weights and [activation functions](#) to capture important patterns, progressively reducing the data's size and complexity.
- **Output(Latent Space):** The encoder outputs a compressed vector known as the latent representation or encoding. This vector captures the important features of the input data in a condensed form helps in filtering out noise and redundancies.

2. Bottleneck (Latent Space)

It is the smallest layer of the network which represents the most compressed version of the input data. It serves as the information bottleneck which force the network to prioritize the most significant features. This compact representation helps the model learn the underlying structure and key patterns of the input helps in enabling better generalization and efficient data encoding.

3. Decoder

It is responsible for taking the compressed representation from the latent space and reconstructing it back into the original data form.

- **Hidden Layers:** These layers progressively expand the latent vector back into a higher-dimensional space. Through successive transformations decoder attempts to restore the original data shape and details

- **Output Layer:** The final layer produces the reconstructed output which aims to closely resemble the original input. The quality of reconstruction depends on how well the encoder-decoder pair can minimize the difference between the input and output during training.

Loss Function in Autoencoder Training

During training an autoencoder's goal is to minimize the reconstruction loss which measures how different the reconstructed output is from the original input. The choice of loss function depends on the type of data being processed:

- **Mean Squared Error (MSE):** This is commonly used for continuous data. It measures the average squared differences between the input and the reconstructed data.
- **Binary Cross-Entropy:** Used for binary data (0 or 1 values). It calculates the difference in probability between the original and reconstructed output.

During training the network updates its weights using [backpropagation](#) to minimize this reconstruction loss. By doing this it learns to extract and retain the most important features of the input data which are encoded in the latent space.

Efficient Representations in Autoencoders

Constraining an autoencoder helps it learn meaningful and compact features from the input data which leads to more efficient representations. After training only the encoder part is used to encode similar data for future tasks. Various techniques are used to achieve this as follows:

- **Keep Small Hidden Layers:** Limiting the size of each hidden layer forces the network to focus on the most important features. Smaller layers reduce redundancy and allows efficient encoding.
- **Regularization:** Techniques like [L1 or L2 regularization](#) add penalty terms to the loss function. This prevents overfitting by removing excessively large weights which helps in ensuring the model to learn general and useful representations.
- **Denoising:** In denoising autoencoders random noise is added to the input during training. It learns to remove this noise during reconstruction which helps it focus on core, noise-free features and helps in improving robustness.
- **Tuning the Activation Functions:** Adjusting activation functions can promote sparsity by activating only a few neurons at a time. This sparsity reduces model complexity and forces the network to capture only the most relevant features.

❖ Types of Autoencoders

Lets see different types of Autoencoders which are designed for specific tasks with unique features:

1. Denoising Autoencoder

Denoising Autoencoder is trained to handle corrupted or noisy inputs, it learns to remove noise and helps in reconstructing clean data. It prevent the network from simply memorizing the input and encourages learning the core features.

2. Sparse Autoencoder

Sparse Autoencoder contains more hidden units than input features but only allows a few neurons to be active simultaneously. This sparsity is controlled by zeroing some hidden units, adjusting activation functions or adding a sparsity penalty to the loss function.

3. Variational Autoencoder

Variational autoencoder (VAE) makes assumptions about the probability distribution of the data and tries to learn a better approximation of it. It uses stochastic gradient descent to optimize and learn the distribution of latent variables. They used for generating new data such as creating realistic images or text.

It assumes that the data is generated by a Directed Graphical Model and tries to learn an approximation to $q_{\phi}(z | x)$ to the conditional property $q_{\theta}(z | x)$ where ϕ and θ are the parameters of the encoder and the decoder respectively.

4. Convolutional Autoencoder

Convolutional autoencoder uses convolutional neural networks (CNNs) which are designed for processing images. The encoder extracts features using convolutional layers and the decoder reconstructs the image through **deconvolution** also called as upsampling.

5. Contractive Autoencoder

- Adds a **regularization penalty** to the loss function.
- Encourages the encoder to learn representations that are **less sensitive to small input changes**.
- Improves robustness of learned features.

❖ Markov Networks (Markov Random Fields)

A **Markov Network**, also called a **Markov Random Field (MRF)**, is an **undirected probabilistic graphical model** used to represent the relationships between random variables. It models the **joint probability distribution** of a set of variables using an **undirected graph**.

In a Markov Network, **nodes represent random variables** and **edges represent probabilistic dependencies** between the variables.

1. Definition

A **Markov Network** is a graphical model where:

- Each **node** represents a random variable.
- **Edges are undirected**, showing mutual relationships between variables.
- The model represents the **joint probability distribution** over all variables.

2. Structure of Markov Network

Components of a Markov Network:

1. **Nodes (Vertices)**
Represent random variables.
2. **Edges (Undirected)**
Show dependency between variables.
3. **Cliques**
A **clique** is a subset of nodes where every node is connected to every other node.
4. **Potential Functions (Clique Potentials)**
Each clique has a **potential function** that represents compatibility between variables.

3. Joint Probability Distribution

The joint probability distribution in a Markov Network is given by:

$$P(X) = \frac{1}{Z} \prod_{c \in C} \psi_c(X_c)$$

Where:

- X = Set of all variables
- C = Set of cliques in the graph
- ψ_c = Potential function for clique c
- Z = Partition function (normalization constant)

The **partition function** Z ensures that the probabilities sum to 1.

4. Markov Property

The **Markov property** states:

A node is conditionally independent of all other nodes given its neighboring nodes.

This means the **probability of a variable depends only on its neighbors**.

5. Example

Consider three variables:

- A = Weather
- B = Traffic
- C = Travel Time

Graph structure:

A ---- B ---- C

Here:

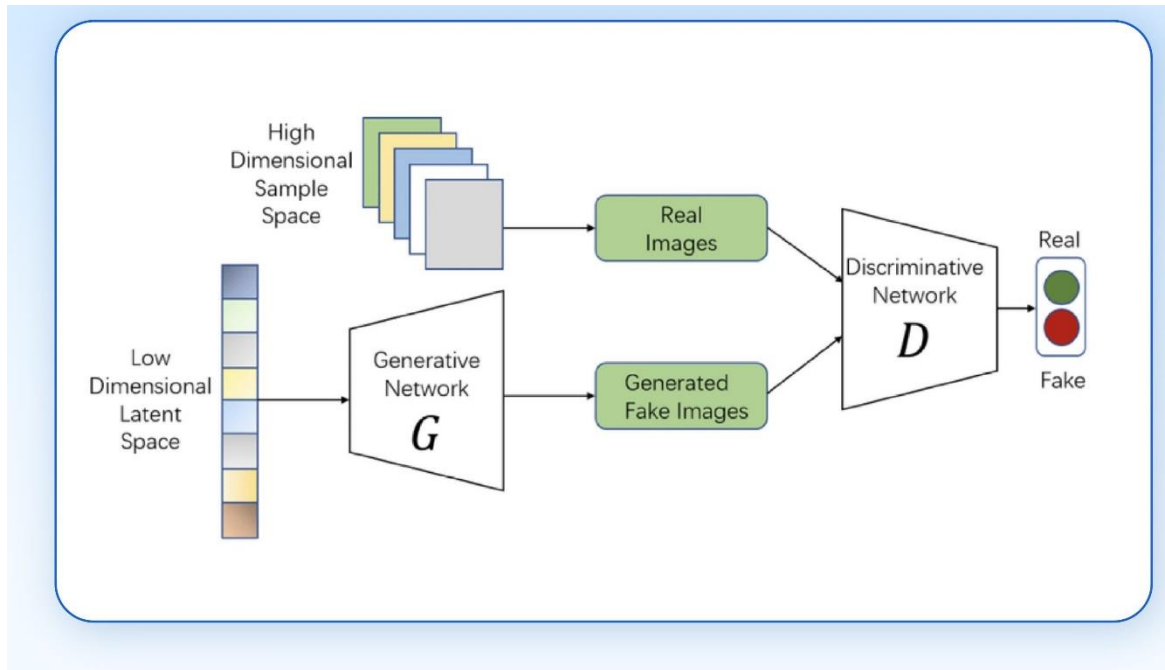
- Weather affects traffic.
- Traffic affects travel time.
- Weather and travel time interact through traffic.

Applications of Markov Networks

- Image processing and computer vision
- Natural language processing
- Pattern recognition
- Bioinformatics
- Social network analysis

❖ Generative Adversarial Network (GAN)

Generative Adversarial Networks (GAN) help machines to create new, realistic data by learning from existing examples. It is introduced by Ian Goodfellow and his team in 2014 and they have transformed how computers generate images, videos, music and more.



Unlike traditional models that only recognize or classify data, they take a creative way by generating entirely new content that closely resembles real-world data. This ability helped various fields such as art, gaming, healthcare and data science. In this article, we will see more about GANs and its core concepts.

Architecture of GAN

GAN consist of two main models that work together to create realistic synthetic data which are as follows:

1. Generator Model

The generator is a deep neural network that takes random noise as input to generate realistic data samples like images or text. It learns the underlying data patterns by adjusting its internal parameters during training through [backpropagation](#). Its objective is to produce samples that the discriminator classifies as real.

Generator Loss Function: The generator tries to minimize this loss:

$$J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i))$$

where

- J_G measure how well the generator is fooling the discriminator.

- $G(z_i)$ is the generated sample from random noise z_i
- $D(G(z_i))$ is the discriminator's estimated probability that the generated sample is real.

The generator aims to maximize $D(G(z_i))$ meaning it wants the discriminator to classify its fake data as real (probability close to 1).

2. Discriminator Model

The discriminator acts as a binary classifier helps in distinguishing between real and generated data. It learns to improve its classification ability through training, refining its parameters to detect fake samples more accurately. When dealing with image data, the discriminator uses convolutional layers or other relevant architectures which help to extract features and enhance the model's ability.

Discriminator Loss Function: The discriminator tries to minimize this loss:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- J_D measures how well the discriminator classifies real and fake samples.
- x_i is a real data sample.
- $G(z_i)$ is a fake sample from the generator.
- $D(x_i)$ is the discriminator's probability that x_i is real.
- $D(G(z_i))$ is the discriminator's probability that the fake sample is real.

The discriminator wants to correctly classify real data as real (maximize $\log D(x_i)$) and fake data as fake (maximize $\log(1 - D(G(z_i)))$)

MinMax Loss

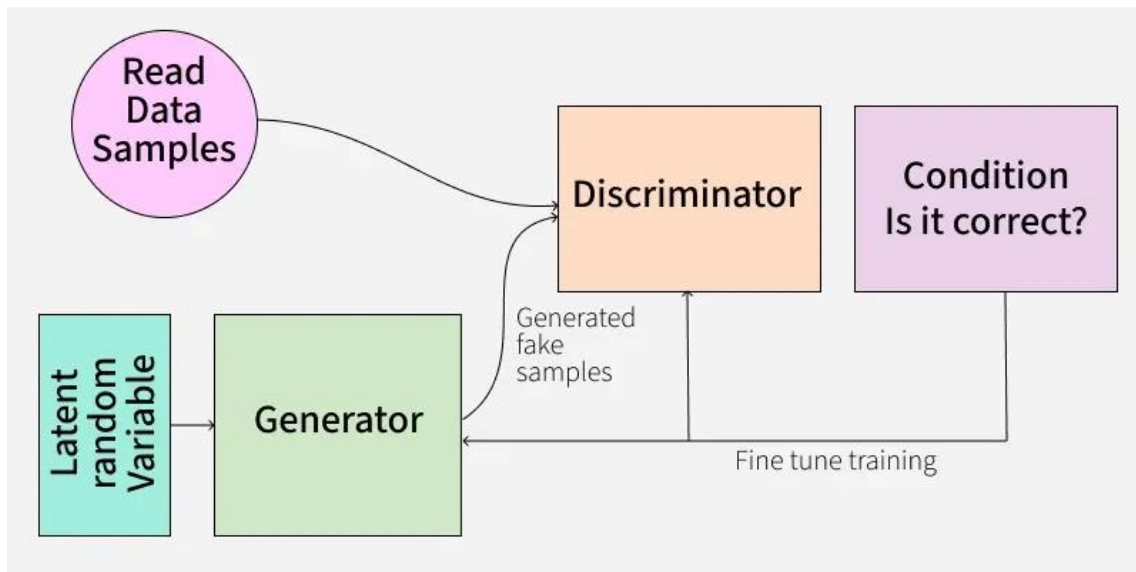
GANs are trained using a [MinMax Loss](#) between the generator and discriminator:

$$\min_G \max_D (G, D) = [\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

where,

- G is generator network and D is the discriminator network
- $p_{data}(x)$ = true data distribution
- $p_z(z)$ = distribution of random noise (usually normal or uniform)
- $D(x)$ = discriminator's estimate of real data
- $D(G(z))$ = discriminator's estimate of generated data

The generator tries to minimize this loss (to fool the discriminator) and the discriminator tries to maximize it (to detect fakes accurately).



How does a GAN work?

GAN train by having two networks the Generator (G) and the Discriminator (D) compete and improve together. Here's the step-by-step process

1. Generator's First Move

The generator starts with a random noise vector like random numbers. It uses this noise as a starting point to create a fake data sample such as a generated image. The generator's internal layers transform this noise into something that looks like real data.

2. Discriminator's Turn

The discriminator receives two types of data:

- Real samples from the actual training dataset.
- Fake samples created by the generator.

D's job is to analyze each input and find whether it's real data or something G cooked up. It outputs a probability score between 0 and 1. A score of 1 shows the data is likely real and 0 suggests it's fake.

3. Adversarial Learning

- If the discriminator correctly classifies real and fake data it gets better at its job.
- If the generator fools the discriminator by creating realistic fake data, it receives a positive update and the discriminator is penalized for making a wrong decision.

4. Generator's Improvement

- Each time the discriminator mistakes fake data for real, the generator learns from this success.
- Through many iterations, the generator improves and creates more convincing fake samples.

5. Discriminator's Adaptation

- The discriminator also learns continuously by updating itself to better spot fake data.
- This constant back-and-forth makes both networks stronger over time.

6. Training Progression

- As training continues, the generator becomes highly proficient at producing realistic data.
- Eventually the discriminator struggles to distinguish real from fake shows that the GAN has reached a well-trained state.
- At this point, the generator can produce high-quality synthetic data that can be used for different applications.

Types of Generative Adversarial Networks (GAN)

1. Vanilla GAN (Basic GAN)

Vanilla GAN is the **original GAN architecture**.

Working

- The **generator** takes random noise as input and generates fake data.
- The **discriminator** receives both real data and generated data.
- The discriminator learns to distinguish between real and fake samples.
- The generator improves by trying to fool the discriminator.

Objective Function

GAN uses a **minimax game** between generator and discriminator.

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

Limitations

- Training instability
- Mode collapse
- Vanishing gradients

Applications

- Basic image generation
- Learning simple data distributions

2. Deep Convolutional GAN (DCGAN)

DCGAN improves the original GAN by using **Convolutional Neural Networks (CNNs)**.

Key Features

- Uses **convolution layers** instead of fully connected layers.
- Uses **batch normalization** for stable training.
- Uses **Leaky ReLU activation** in discriminator.
- Uses **transposed convolution** for upsampling in generator.

Advantages

- Generates **higher quality images**
- Better **training stability**

Applications

- Image generation
- Image feature learning
- Image synthesis

3. Conditional GAN (CGAN)

Conditional GAN generates data **based on specific conditions or labels**.

Concept

Both the generator and discriminator receive **additional information (y)** such as:

- Class labels
- Attributes
- Text descriptions

Objective Function

$$G(z \mid y), D(x \mid y)$$

Where:

- y = condition or label.

Example

- Generate images of a **specific digit** (0–9).
- Generate images of **specific objects**.

Applications

- Controlled image generation
 - Text-to-image generation
 - Data augmentation
-

4. Cycle GAN

CycleGAN is used for **image-to-image translation without paired data**.

Structure

CycleGAN consists of:

- **Two generators**
- **Two discriminators**

Example:

- Generator G : Domain A $\rightarrow$ Domain B
- Generator F : Domain B $\rightarrow$ Domain A

Cycle Consistency

If an image is converted from A $\rightarrow$ B and back B $\rightarrow$ A, it should produce the **original image**.

$$F(G(x)) \approx x$$

Applications

- Convert **horses to zebras**
 - Convert **summer images to winter images**
 - Artistic style transfer
-

5. Style GAN

StyleGAN is a **state-of-the-art GAN architecture** developed by researchers at NVIDIA.

Key Idea

Instead of feeding noise directly into the generator, StyleGAN introduces a **style-based generator architecture**.

Features

- Controls image **style at different levels**
- Produces **very realistic human faces**
- Allows manipulation of attributes such as:
 - Hair style
 - Age
 - Facial expressions

Applications

- Face generation
 - Synthetic image creation
 - Media and graphics
-

❖ Applications

- **Image Synthesis & Generation:** GANs generate realistic images, avatars and high-resolution visuals by learning patterns from training data. They are used in art, gaming and AI-driven design.
- **Image-to-Image Translation:** They can transform images between domains while preserving key features. Examples include converting day images to night, sketches to realistic images or changing artistic styles.
- **Text-to-Image Synthesis:** They create visuals from textual descriptions helps applications in AI-generated art, automated design and content creation.
- **Data Augmentation:** They generate synthetic data to improve machine learning models helps in making them more robust and generalizable in fields with limited labeled data.
- **High-Resolution Image Enhancement:** They upscale low-resolution images which helps in improving clarity for applications like medical imaging, satellite imagery and video enhancement.

❖ Advantages

Lets see various advantages of the GANs:

- **Synthetic Data Generation:** GANs produce new, synthetic data resembling real data distributions which is useful for augmentation, anomaly detection and creative tasks.
- **High-Quality Results:** They can generate photorealistic images, videos, music and other media with high quality.
- **Unsupervised Learning:** They don't require labeled data helps in making them effective in scenarios where labeling is expensive or difficult.
- **Versatility:** They can be applied across many tasks including image synthesis, text-to-image generation, style transfer, anomaly detection and more.

❖ Challenges Faced by GANs

Generative Adversarial Networks (GANs) are powerful deep learning models used to generate realistic data such as images, audio, and text. However, training GANs is difficult and involves several challenges.

1. Mode Collapse

- The generator produces **limited varieties of outputs**, even though the real dataset has many variations.
- Example: When generating faces, the model may produce many **similar-looking faces** instead of diverse ones.
- This happens when the generator finds a few outputs that consistently fool the discriminator.

2. Training Instability

- GAN training involves two networks: **Generator and Discriminator** competing with each other.
- If one network becomes **too strong**, the other cannot learn effectively.
- This can cause **oscillations or failure to converge**.

3. Vanishing Gradients

- If the discriminator becomes **too accurate**, it easily distinguishes fake from real data.
- As a result, the generator receives **very small gradient updates**, making learning slow or stopping learning entirely.

4. Difficulty in Convergence

- Unlike traditional neural networks, GANs do not have a **clear loss minimization objective**.

- It is hard to determine when the model has reached the **optimal equilibrium**.

5. Evaluation Difficulty

- Measuring the quality of generated data is difficult.
- Metrics such as **Inception Score (IS)** and **Fréchet Inception Distance (FID)** are used, but they may not fully reflect visual quality.

6. High Computational Cost

- GANs require **large datasets, powerful GPUs, and long training times**.
- Training complex GAN architectures can be computationally expensive.

7. Sensitivity to Hyperparameters

- GAN performance depends heavily on parameters such as:
 - Learning rate
 - Network architecture
 - Batch size
- Small changes can significantly affect the results.

8. Overfitting of Discriminator

- If the discriminator memorizes the training data, it may not generalize well.
- This reduces the ability of the generator to learn meaningful patterns.